2 Java modules

This chapter covers

- Java's platform modules
- Changes to access control semantics
- Writing modular applications
- Multi-release JARs

As mentioned in chapter 1, versions of Java, up to and including Java 9, were delivered according to a feature-driven release plan, often with a major new capability that defined or was strongly associated with the release.

For Java 9, this feature was Java Platform Modules (also known as JPMS, Jigsaw, or just "modules"). This is a major enhancement and change to the Java platform that had been discussed for many years—it was originally conceived of as potentially shipping as a part of Java 7, back in 2009/2010.

In this chapter, we will explain the reasons modules are needed, as well as the new syntax used to articulate modularity concepts and how to use them in your applications. This will enable you to use JDK and third-party modules in your build as well as packaging apps or libraries as modules.

NOTE Modules represent a new way of packaging and deploying your code, and adopting them will make your applications better. However, if you just want to start using modern Java features (11 or 17), there is no need for you to adopt modules immediately unless you want to.

The arrival of modules has profound implications for the architecture of applications, and modules have many benefits to modern projects that are concerned about such aspects as process footprint, startup cost, and warmup time. Modules can also help to solve the so-called JAR Hell problem that can plague Java applications with complex dependencies. Let's get to know them.

2.1 Setting the scene

A *module* is a fundamentally new concept in the Java language (as of Java 9). It is a unit of application deployment and dependency that has semantic meaning to the runtime. This is different from existing concepts in Java for the following reasons:

- JAR files are invisible to the runtime—they're basically just zipped directories containing class files.
- Packages are really just namespaces to group classes together for access control.
- Dependencies are defined at the class level only.
- Access control and reflection combine in a way that produces a fundamentally open system without clear deployment unit boundaries and with minimal enforcement.

Modules, on the other hand

- Define dependency information between modules, so all sorts of resolution and linkage problems can be detected at compile or application start time
- Provide proper encapsulation, so internal packages and classes can be made safe from pesky users who might want to fiddle with them
- Are a proper unit of deployment with metadata that can be understood and consumed by a modern Java runtime and are represented in the Java type system (e.g., reflectively)

NOTE Before modules, within the core language and runtime environment, there was no aggregated dependency metadata. Instead, it was defined only in build systems like Maven or in third-party modules systems (such as OSGI or JBoss modules) that the JVM neither knows nor cares about.

Java platform modules represent an implementation of a missing concept within the Java world as it existed at version 8.

NOTE Java modules are often packaged as special JAR files, but they are not tied to that format (we will see other possible formats later).

The aim of the modules system is to make the deployment units (modules) as independent of each other as possible. The idea is that modules are able to be separately loaded and linked, although in practice, real applications may well end up depending on a group of modules that provide related capabilities (such as security).

1.1 Project Jigsaw

The project within OpenJDK to deliver the modules feature was known as *Project Jigsaw*. It aimed to deliver a full-featured modularity solution which included the following goals:

- Modularizing the JDK platform source
- Reducing the process footprint
- Improving the application startup time
- Having modules available to both the JDK and to application code
- Allowing true strict encapsulation for the first time in Java
- Adding new, previously impossible access control modes to the Java language

These goals were, in turn, driven by the following other objectives that are more closely focused on the JDK and Java runtime:

- Bringing the end of a single, monolithic runtime JAR (`rt.jar`)
- Properly encapsulating and protecting JDK internals
- Allowing major internal changes to be made (including changes that will break unauthorized non-JDK usage)
- Introducing modules as "super packages"

These secondary goals may require a bit more explanation because they are more closely connected to internal and implementation aspects of the platform.

Modular, not monolithic Java runtime

The legacy JAR format is essentially just a zip file that contains classes. It dates back to the earliest days of the platform and is in no way optimized for Java classes and applications. Abandoning the JAR format for the platform classes can help in several areas—for example, enabling much better startup performance.

Modules provide two new formats—*JMOD* and *JIMAGE*—which are used at different times (compile/link time and runtime, respectively) in the program lifecycle.

The JMOD format is somewhat similar to the existing JAR format, but it has been modified to allow the inclusion of native code as part of a single file (rather than having to ship a separate shared object file as is done in Java 8). For most developer needs, including publishing modules to Maven, it's better to package your own modules as modular JARs rather than as JMODs.

The JIMAGE format is used to represent a Java runtime image. Until Java 8, only two possible runtime images existed (JDK and JRE), but this was largely an accident of history. Oracle introduced the *Server JRE* with Java 8 (as well as *Compact Profiles*) as a stepping-stone toward full modularity. These images basically removed some capabilities (e.g., GUI frameworks) to provide a smaller footprint specifically geared toward the needs of server-side applications.

A modular application has enough metadata that the exact set of dependencies can be known before program start. This leads to the possibility that only what is needed has to be loaded, which is much more efficient. It is possible to go even further and define a *custom runtime image* that can be shipped along with an application and that does not contain a full, general-purpose installation of Java but only what the application requires. We will encounter this last possibility at the end of this chapter when we meet the `jlink` tool.

For now, let's meet the `jimage` tool that's available to show details about a Java runtime image. For example, for a Java 15 full runtime (i.e., what used to be contained in a JDK), see the following code sample:

```
$ jimage info $JAVA_HOME/lib/modules
 Major Version:  1
 Minor Version:  0
 Flags:          0
 Resource Count: 32780
 Table Length:   32780
 Offsets Size:   131120
 Redirects Size: 131120
 Locations Size: 680101
 Strings Size:   746471
 Index Size:     1688840
```

or

```
$ jimage list $JAVA_HOME/lib/modules
jimage: /Library/Java/JavaVirtualMachines/java15/Contents/Home/lib/modules

Module: java.base
    META-INF/services/java.nio.file.spi.FileSystemProvider
    apple/security/AppleProvider$1.class
    apple/security/AppleProvider$ProviderService.class
    apple/security/AppleProvider.class
    apple/security/KeychainStore$CertKeychainItemPair.class
```

```
apple/security/KeychainStore$KeyEntry.class
apple/security/KeychainStore$TrustedCertEntry.class
apple/security/KeychainStore.class
com/sun/crypto/provider/AESCipher$AES128_CBC_NoPadding.class
... many, many lines of output
```

Moving away from `rt.jar` allows for better startup performance and to optimize for only what is needed by an application. The new formats are designed to be opaque to the developer and are implementation-dependent. It is no longer possible to just unzip `rt.jar` and get back the JDK's class library. This is just one step, however, in making the platform's internals less accessible to Java programmers, which was one of the goals of the modules system.

Encapsulate the internals

The contract between the Java platform and its users was always intended to be an API contract—that backward compatibility would be maintained at the interface level, *not* in the details of the implementation.

However, Java developers have not held up their end of the bargain and, instead, over time, have tended to use parts of the platform implementation that were never intended for public consumption.

This is problematic, because the OpenJDK platform developers want the freedom to modify the implementation of the JVM and platform classes to future-proof and modernize them—to provide new features and better performance without worrying about breaking user applications.

One major impediment to making breaking changes to the platform internals is Java's approach to access control as it exists in Java 8. Java only defines `public`, `private`, `protected`, and `package-private` as access control levels, and these modifiers are applied only at the class level and finer.

We can work around these restrictions in numerous ways (such as reflection or creating additional classes in relevant packages), and there is no foolproof (or expert-proof) way to fully protect the internals.

The use of the workarounds to access the internals was historically often for valid reasons. As the platform has matured, however, an official way of accessing almost all of the desired functionality has been added. The unprotected internals, therefore, repre-

sent a liability for the platform going forward without a corresponding benefit—and modularity was one way to remove that legacy problem.

To sum up, Project Jigsaw was a way to solve several problems at once—primarily to reduce runtime size, improve startup time, and tidy up dependencies between internal packages. These were problems that were hard (or impossible) to tackle incrementally. Opportunities for these types of "nonlocal" improvement do not come along very often, especially in mature software platforms, so the Jigsaw team wanted to take advantage of their circumstances.

The JVM is modular now

To see this, consider the next very simple program:

```
public class StackTraceDemo {
   public static void main(String[] args) {
      var i = Integer.parseInt("Fail");
   }
}
```

Compiling and running this code produces a runtime exception, shown next:

```
$ java StackTraceDemo
Exception in thread "main" java.lang.NumberFormatException:
  For input string: "Fail"
    at java.base/java.lang.NumberFormatException.forInputString(
    NumberFormatException.java:65)
    at java.base/java.lang.Integer.parseInt(Integer.java:652)
    at java.base/java.lang.Integer.parseInt(Integer.java:770)
    at StackTraceDemo.main(StackTraceDemo.java:3)
```

However, we can clearly see that the format of the stack trace has changed somewhat from the form that was used in Java 8. In particular, the stack frames are now qualified by a module name (`java.base`) as well as a package name, class name, and line number. This clearly shows that the modular nature of the platform is pervasive and is present for even the simplest program.

1.2 The module graph

Key to all of modularity is the *module graph*, which is a representation of how modules depend on each other. Modules make their dependencies explicit via some new syntax, and those dependencies are hard guarantees that the compiler and runtime can rely upon. One very important concept is that the module graph must be a *directed acyclic graph* (DAG), so in mathematical terms, there cannot be any cyclic dependencies.

NOTE It is important to realize that in modern Java environments, all applications run on top of the modular JRE; a "modular mode" and a "legacy classpath mode" don't exist.

Although not every developer needs to become an expert in the modules system, it makes sense that a well-grounded Java developer would benefit from a working knowledge of a new subsystem that has changed the way that all programs are executed on the JVM. Let's take a look at a first view of the modules system, shown in figure 2.1, as most developers encounter it.

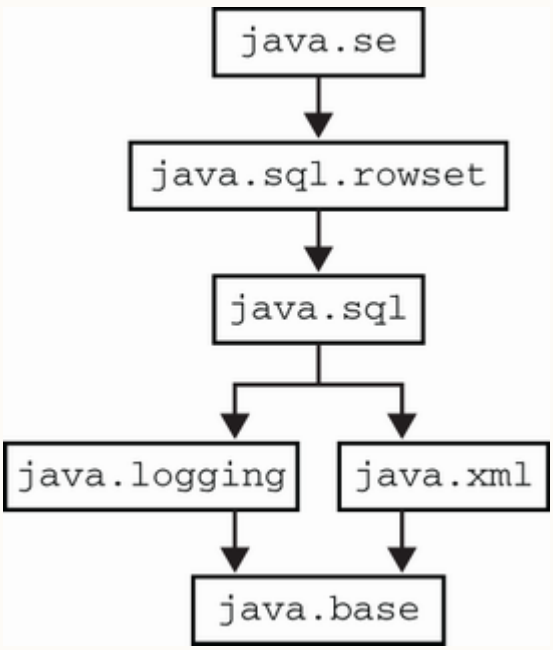


Figure 2.1 JDK system modules (simplified view)

In figure 2.1, we can see a simplified view of some of the main modules in the JDK. Note that the module `java.base` is always a dependency of every module. When drawing pictures of module graphs, the implicit dependency on `java.base` is often eliminated just to reduce visual clutter.

The clean and relatively simple set of module boundaries that we can see in figure 2.1 needs to be contrasted with the state of the JDK in Java 8. Unfortunately, before modules, Java's top-level unit of code was the package—and Java 8 had almost 1,000 of them in the standard runtime. This would be essentially impossible to draw, and the de-

pendencies within the graph would be so complex that a human would not be able to make sense of it.

Taking the premodular JDK and reshaping it into the well-defined form that we see today was not easy to achieve, and the path to delivering JDK modularity was long. Java 9 was released in September 2017, but development of the feature had begun several years before that with the Java 8 release train. In particular, there were several subgoals that were necessary first steps for the delivery of modules, including the following:

- Modularizing the layout of source code in the JDK (JEP 201)
- Modularizing the structure of runtime images (JEP 220)
- Disentangling complex implementation dependencies between JDK packages

Even though the finished modules feature did not ship until Java 9, much of the cleanup was undertaken as part of Java 8 and even allowed a feature known as *compact profiles* (which we will meet at the end of this chapter) to ship as part of that release.

1.3 Protecting the internals

One of the major problems that modules needed to solve was overcoupling of user Java frameworks to internal implementation details. For example, this piece of Java 8 code extends an internal class to get access to a low-level *URL canonicalizer*.

The following code is for demonstration purposes only, so we can have a concrete example to discuss modules and access control—your code should never access internal classes directly:

```
import sun.net.URLCanonicalizer;

public class MyURLHandler extends URLCanonicalizer {

   public boolean isSimple(String url) {
      return isSimpleHostName(url);
   }
}
```

A URL canonicalizer is a piece of code that takes a URL in one of the various forms permitted by the URL standard and converts it to a standard (canonical) form. The intent is that canonical URLs can act as a single source of truth for the location of content that can be accessed via multiple different possible URLs. If we try to compile it using Java 8, `javac` warns us that we're accessing an internal API, as shown next:

```
$ javac MyURLHandler.java
MyURLHandler.java:1: warning: URLCanonicalizer is internal proprietary API
 and may be removed in a future release

import sun.net.URLCanonicalizer;
           ^
MyURLHandler.java:3: warning: URLCanonicalizer is internal proprietary API
 and may be removed in a future release

public class MyURLHandler extends URLCanonicalizer {
                          ^
2 warnings
```

However, by default, the compiler still allows access, and the result is a user class that is tightly coupled to the internal implementation of the JDK. This connection is fragile and will break if the called code moves or is replaced.

If enough developers abuse this openness, then this leads to a situation in which it is difficult or impossible to make changes to the internals, because to do so would break deployed libraries and applications.

NOTE The `URLCanonicalizer` class needs to be called from several different packages, not just its own, so it has to be a public class—it can't be package-private—meaning it's accessible to anyone.

The solution to this very general problem was to make a one-time change to Java's model of access control. This change applies both to user code calling the JDK and to applications calling third-party libraries.

1.4 New access control semantics

Modules add a new concept to Java's access control model: the idea of *exporting* a package. In Java 8 and earlier, code in any package can call public methods on any public class in any package. This is sometimes called "shotgun privacy," after a famous quote about another programming language:

Perl doesn't have an infatuation with enforced privacy. It would prefer that you stayed out of its living room because you weren't invited, not because it has a shotgun.

—Larry Wall

For Java, however, shotgun privacy represented a major problem. More and more libraries were using internal APIs to provide capabilities that were difficult or impossible to provide in another way, and this threatened to harm the long-term health of the platform.

As of Java 8, there was no way to enforce access control across an entire package. This meant that the JDK team was unable to define a public API and know with certainty that clients of that API could not subvert it or directly link to the internal implementation.

The convention that anything in a package that starts `java` or `javax` is a public API and everything else is internal only is just that—a convention. No VM or class loading mechanism enforces that, as we've already seen.

With modules, however, this changes. The `exports` keyword has been introduced to indicate which packages are considered the public API of a module. In the modular JDK, the package `sun.net` is not exported, so the previous Java 8 URL canonicalizer code will not compile. Here's what happens when we try with Java 11:

```
$ javac src/ch02/MyURLHandler.java
src/ch02/MyURLHandler.java:3: error: package sun.net is not visible
import sun.net.URLCanonicalizer;
       ^
  (package sun.net is declared in module java.base, which does not export
    it to the unnamed module)
src/ch02/MyURLHandler.java:8: error: cannot find symbol
      return isSimpleHostName(url);
             ^
  symbol:   method isSimpleHostName(String)
  location: class MyURLHandler
2 errors
```

Note that the form of the error message explicitly says that the `sun.net` package is now not visible—the compiler cannot even see the symbol. This is a fundamental change in the way Java access control works. Only methods on exported packages are accessible. It is no longer the case that a public method on a public class is automatically visible to all code everywhere.

However, this change may not be visible to many developers. If you're a Java developer who plays by the rules, you will never have called an API in an internal package directly. However, you might use a library or a framework that does, so it's good to understand what has actually changed and avoid the FUD.

NOTE Proper encapsulation is not free, and premodular Java is actually a very open system. It is perhaps only natural that when confronted with the more structured system that modules provide, many Java developers find some of the extra protections constraining or frustrating. Let's meet the syntax that encodes these new semantics of Java modules.

2.2 Basic modules syntax

A Java platform module is defined as a conceptual unit, which is a collection of packages and classes that are declared and loaded as a single entity. Each module must declare a new file, called a *module descriptor*, represented as a module-info.java file, which contains the following:

- Module name
- Module dependencies
- Public API (packages exported)
- Reflective access permissions
- Services provided
- Services consumed

This file must be placed in a suitable place within the source hierarchy. For example, within a Maven-style layout, the full module name `wgjd.discovery` appears directly after src/main/java and contains module-info.java and the package root, as shown here:

```
src
└── main
    └── java
        └── wgjd.discovery
            ├── wgjd
            │   └── discovery
            │       ├── internal
            │       │   ├── AttachOutput.java
            │       │   └── PlainAttachOutput.java
            │       ├── VMIntrospector.java
            │       └── Discovery.java
            └── module-info.java
```

This is, of course, slightly different from nonmodular Java projects, which often nominate src/main/java as the root of the package directories. However, the familiar hierarchical structure of packages under the module root is still visible.

NOTE When a modular project is built, the module descriptor will be compiled into a class file, module-info.class, but that file (despite its name) is actually quite different from the usual sort of class file that we see in the Java platform.

In this chapter we will address the basic directives of the descriptor but will not be delving deeply into all of the capabilities that modules provide. In particular, we will not discuss the services aspects of modules.

A simple example of a module descriptor looks like this:

```
module wgjd.discovery {
  exports wgjd.discovery;

  requires java.instrument;
  requires jdk.attach;
  requires jdk.internal.jvmstat;
}
```

This contains three new keywords— `module`, `exports`, and `requires` —in a syntax that should be suggestive to most Java programmers. The keyword `module` simply declares the opening scope of the declaration.

NOTE The name module-info.java is reminiscent of package-info.java, and they are somewhat related. Because packages are not really visible to the runtime, a workaround (hack?) was needed to provide a hook for annotation metadata that was intended to apply to the whole package. This hack was `package-info.java`. In the modular world, much more metadata can be associated with a module, and so a similar name was chosen. The new syntax actually consists of *restricted keywords*, which are described in the Java Language Specification like this:

A further ten character sequences are restricted keywords: `open`, `module`, `requires`, `transitive`, `exports`, `opens`, `to`, `uses`, `provides`, *and* `with`. *These character sequences are tokenized as keywords solely where they appear as terminals in the* `ModuleDeclaration` *and* `ModuleDirective` *productions.*

In simpler language, this means these new keywords will appear only in the descriptor for the module metadata and are not treated as keywords in general Java source. However, it is good practice to avoid these words as Java identifiers, even if it is technically legal to use them. This is the same situation as we saw with `var` in chapter 1, and

we will use looser language and refer to them as “keywords” throughout the rest of the book.

2.1 Exporting and requiring

The `exports` keyword expects an argument, which is a package name. In our example

```
exports wgjd.discovery;
```

means that our example discovery module exports the package `wgjd.discovery`, but because the descriptor does not mention any other packages, `wgjd.discovery.internal` is not exported and is not normally available to code outside the discovery module.

Multiple `exports` lines are possible in a module descriptor and, in fact, are quite usual. Fine-grained control is also possible with the `exports ... to ...` syntax that indicates only certain external modules may access a specified package from this module.

NOTE A single module `exports` one or more packages that constitute the public API of the module and that are the only packages that code in other modules may access unless an override (e.g., command-line switch) is used.

The `requires` keyword declares a dependency of the current module and always requires an argument, which is a *module* name, rather than a package name. The `java.base` module contains the most fundamental packages and classes of the Java runtime. We can use the `jmod` command to take a look, as follows:

```
$ jmod describe $JAVA_HOME/jmods/java.base.jmod
java.base@11.0.3
exports java.io
exports java.lang
exports java.lang.annotation
exports java.lang.invoke
exports java.lang.module
exports java.lang.ref
exports java.lang.reflect
exports java.math
exports java.net
exports java.net.spi
exports java.nio
// ... many, many more lines of output
```

These packages are used by every Java program, and so `java.base` is always an implicit dependency of every module, so does not need to be explicitly declared in `module-info.java`. This is in much the same way that `java.lang` is an implicit `import` into every Java class.

Some of the basic rules and conventions for module names follow:

- Modules live in a global namespace.
- Module names must be unique.
- Use the standard `com.company.project` convention if appropriate.

One important basic modules concept is *transitivity*. Let's take a closer look at this concept, because it occurs not only in the context of modules but also in Java's more familiar library (i.e., JAR file) dependencies (which we will meet in chapter 11).

2.2 Transitivity

Transitivity is a very general computing term, not specific to Java at all, which describes the situation that occurs when a code unit requires other units to function correctly, and those units can themselves require other units. Our original code may never even mention these "one step removed" code units, but they still need to be present or our application will not work.

To understand why this is the case—and why it is important—consider two modules `A` and `B` where `A` requires `B`. There are two different possible cases:

- `A` does not export any methods that mention types from `B` directly.
- `A` includes types from `B` as part of its API.

In the case where `A` exports methods that return types that are defined in `B`, this would have the effect that `A` is not usable unless clients of `A` (those modules that require `A`) also require `B`. This is quite an unnecessary overhead on clients of `A`.

The modules system provides some simple syntax to solve this: `requires transitive`. If a module `A` requires another module transitively, then any code that depends on `A` will also, implicitly, pick up the transitive dependencies as well.

Although usage of `requires` `transitive` is unavoidable in some use cases, in general, when writing modules, minimizing use of transitivity is considered a best practice. We will have more to say about transitive dependencies when we discuss build tools in chapter 11.

2.3 Loading modules

If the first time you've encountered Java class loading is when we briefly mentioned it in chapter 1 and you have no other experience of it, don't worry. The most important thing to know right now is that the following four types of modules exist, some of which have slightly different behaviors when loaded:

- Platform modules
- Application modules
- Automatic modules
- Unnamed module

On the other hand, if you're already familiar with class loading, you should know that the arrival of modules has changed some of the details of the way that class loading operates.

A modern JVM has module-aware class loaders, and the way that the JRE classes are loaded is quite different than in Java 8. One key concept is the *module path*, which is a sequence of paths to modules (or directories that contain modules). This is similar to, but separate from, the traditional Java classpath.

NOTE We will meet class loading properly in chapter 4 and introduce the modern way of doing things to both new and experienced readers.

The fundamental principles of the modular approach to class loading follow:

- Modules are resolved from the module path, not the old-school classpath.
- At startup, the JVM resolves a graph of modules, which must be acyclic.
- One module is the root of the graph and is where execution starts from. It contains the class with the main method that will be the entry point.

Dependencies that have already been modularized are known as *application modules* and are placed on the module path. Unmodularized dependencies are placed on the familiar classpath, and are coopted into the modules system via a migration mechanism.

Module resolution uses depth-first traversal, and because the graph must be acyclic, the resolution algorithm will terminate (and in linear time). Let's delve a little more deeply into each of the four types of modules.

3.1 Platform modules

These are modules from the modular JDK itself. They would have been part of the monolithic runtime (`rt.jar`) in Java 8 (or possibly ancillary JARs, such as `tools.jar`). We can get a list of the available platform modules from the `--list-modules` flag, as shown here:

```
$ java --list-modules
java.base@11.0.6
java.compiler@11.0.6
...
java.xml@11.0.6
java.xml.crypto@11.0.6
jdk.accessibility@11.0.6
...
jdk.unsupported@11.0.6
...
```

This code will provide an unabridged list, rather than the partial set that we saw in figure 2.1.

NOTE The exact list of modules and their names will depend on the version of Java in use. For example, on Oracle's GraalVM implementation, some additional modules like `com.oracle.graal.graal_enterprise` , `org.graalvm.js .scriptengine` and `org.graalvm.sdk` may be present.

The platform modules make heavy use of the *qualified exporting* mechanism wherein some packages are exported only to a specified list of modules and are not made generally available.

The most important module in the distribution is `java.base` , which is always an implicit dependency of every other module. It contains `java.lang` , `java.util` , `java.io` , and various other basic packages. The module basically corresponds to the smallest possible Java runtime that an application could require and still run.

At the other end of the spectrum are the *aggregator modules*, which don't contain any code but which serve as a shortcut mechanism to allow applications to bring in a very broad set of dependencies transitively. For example, the `java.se` module brings in the entire Java SE platform.

3.2 Application modules

These types of modules are the modularized dependencies of an application, or the application itself. This type of module is also sometimes known as a *library module*.

NOTE No technical distinction exists between platform and application modules—the difference is purely philosophical—and which class loader is used to load them, as we will discuss in chapter 4.

The third-party libraries that an application depends on will be application modules. For example, the Jackson libraries for manipulating JSON have been modularized as of version 2.10 and count as application (aka library) modules.

Application modules will typically depend upon both platform modules and other application modules. It is a good idea to try to constrain the dependencies of these modules as much as possible and to avoid requiring, for example, `java.se` as a dependency.

3.3 Automatic modules

One deliberate design feature of the modules system is that you can't reference the classpath from a module. This restriction seems to be potentially problematic—what happens if a module needs to depend on some code that has not yet been modularized?

The solution is to move the nonmodular JAR file onto the module path (and remove it from the classpath). When this is done, the JAR becomes an *automatic module*. The modules system will automatically generate a name for your module, which is derived from the JAR's name.

An automatic module exports every package that it contains, and automatically depends upon all other modules in the module path. Automatic modules do not have proper module dependency information, because they neither explicitly declare their dependencies nor advertise their API. This means that they are not first-class citizens in the modules system and do not provide the same level of guarantees as genuine Java modules.

It is possible to explicitly declare a name, by adding an entry for `Automatic-Module-Name` into the MANIFEST.MF file in the JAR. This is often done as an intermediate step when migrating to Java modules, because it allows developers to reserve a module name and start to gain some of the benefits of interoperating with modular code.

For example, the Apache Commons Lang library is not yet fully modularized, but it provides `org.apache.commons.lang3` as an automatic module name. Other modules can

then declare that they depend upon this automatic module, even if the maintainers of it have not finished transitioning it to full modularity.

3.4 Unnamed module

All classes and JARs on the classpath are added to a single module, which is the unnamed module, or `UNNAMED`. This is done for backward compatibility but at the cost that the modules system is not as effective as it might be all the time that some code remains in the unnamed module.

For the case of completely nonmodular apps (e.g., Java 8 apps that are running on top of a Java 11 runtime), the contents of the classpath are dumped into the unnamed module, and the root module is taken to be `java.se`.

Modular code cannot depend on the unnamed module, and so in practice, modules cannot depend upon anything in the classpath. Automatic modules are often used to help resolve this situation. Formally, the unnamed module depends upon all modules in JDK and on the module path because it is replicating the premodular behavior.

2.4 Building a first modular app

Let's build a first example of a modular application. To do this, we need to build a module graph (which is, of course, a DAG). The graph must have a *root module*, which in our case is the module containing the entry point class of the app. The module graph of the application is the transitive closure of all modular dependencies of the root module.

For our example, we're going to adapt the HTTP site-checking tool we created at the end of chapter 1 to become a modular app. The files will be laid out like this:

```
.
└── wgjd.sitecheck
    ├── wgjd
    │   └── sitecheck
    │       ├── concurrent
    │       │   └── ParallelHTTPChecker.java
    │       ├── internal
    │       │   └── TrustEveryone.java
    │       ├── HTTPChecker.java
    │       └── SiteCheck.java
    └── module-info.java
```

We're breaking out certain concerns (e.g., the `TrustEveryone` provider) into their own classes rather than representing them as static inner classes, as we had to when all the code needed to live in a single file. We've also set up separate packages and will not be exporting all of them. The module file is very similar to the one we met earlier, shown here:

```
module wgjd.sitecheck {
  requires java.net.http;
  exports wgjd.sitecheck;
  exports wgjd.sitecheck.concurrent;
}
```

Note the dependency on the module `java.net.http`. To investigate what happens when a dependency is missed, let's comment out the dependency on the HTTP module and try to compile the project using `javac` as follows:

```
$ javac -d out wgjd.sitecheck/module-info.java \
    wgjd.sitecheck/wgjd/sitecheck/*.java \
    wgjd.sitecheck/wgjd/sitecheck/*/*.java
wgjd.sitecheck/wgjd/sitecheck/SiteCheck.java:8: error:
  package java.net.http is not visible
import java.net.http.*;
            ^
  (package java.net.http is declared in module java.net.http, but
     module wgjd.sitecheck does not read it)
wgjd.sitecheck/wgjd/sitecheck/concurrent/ParallelHTTPChecker.java:4:
  error: package java.net.http is not visible
import java.net.http.*;
            ^

// Several similar errors
```

This failure shows that simple problems with modules can be very easy to solve. The modules system has detected the missing module and is trying to help by suggesting a solution: add the missing module as a dependency. If we make that change, then, as expected, the module builds without complaint. However, more complex problems may require changes to the compilation step or manual intervention via a switch to control the modules system.

4.1 Command-line switches for modules

When compiling a module, a number of command-line switches can be used to control the modular aspects of the compile (and, later, execution). The most commonly encountered of these switches follow:

- `list-modules` —Prints a list of all modules
- `module-path` —Specifies one or more directories that contain your modules
- `add-reads` —Adds an additional `requires` to the resolution
- `add-exports` —Adds an additional `exports` to the compilation
- `add-opens` —Enables reflective access to all types at runtime
- `add-modules` —Adds the list of modules to the default set
- `illegal-access=permit|warn|deny` —Changes the reflective access rule

We have already met the majority of these concepts already—with the exception of the qualifiers related to reflection, which we will discuss in detail in section 2.4.3

Let's see one of these switches in action. This will demonstrate a common issue with module packaging and serves as an example of a real-world issue that many developers may encounter when starting to use modules with their own code.

When starting to work with modules, we sometimes find that we need to break encapsulation. For example, an application that has been ported from Java 8 may be expecting to access an internal package that is no longer exported.

For example, let's consider a project with a simple structure that uses the *Attach API* to dynamically connect to other JVMs running on a host and report some basic information about them. It's laid out on disc like this, just as we saw in an earlier example:

```
.
└── wgjd.discovery
    ├── wgjd
    │   └── discovery
    │       ├── internal
    │       │   └── AttachOutput.java
    │       ├── Discovery.java
    │       └── VMIntrospector.java
    └── module-info.java
```

Compiling the project gives the following series of errors:

```
$ javac -d out/wgjd.discovery wgjd.discovery/module-info.java \
  wgjd.discovery/wgjd/discovery/*.java \
  wgjd.discovery/wgjd/discovery/internal/*

wgjd.discovery/wgjd/discovery/VMIntrospector.java:4: error: package
  sun.jvmstat.monitor is not visible
import sun.jvmstat.monitor.MonitorException;
               ^
  (package sun.jvmstat.monitor is declared in module jdk.internal.jvmstat,
    which does not export it to module wgjd.discovery)
wgjd.discovery/wgjd/discovery/VMIntrospector.java:5: error: package
  sun.jvmstat.monitor is not visible
import sun.jvmstat.monitor.MonitoredHost;
               ^
  (package sun.jvmstat.monitor is declared in module jdk.internal.jvmstat,
    which does not export it to module wgjd.discovery)
```

These problems are being caused by some code in the project that makes use of internal APIs, as shown next:

```
public class VMIntrospector implements Consumer<VirtualMachineDescriptor> {

    @Override
    public void accept(VirtualMachineDescriptor vmd) {
        var isAttachable = false;
        var vmVersion = "";
        try {
            var vmId = new VmIdentifier(vmd.id());
            var monitoredHost = MonitoredHost.getMonitoredHost(vmId);
            var monitoredVm = monitoredHost.getMonitoredVm(vmId, -1);
            try {
                isAttachable = MonitoredVmUtil.isAttachable(monitoredVm);
                vmVersion = MonitoredVmUtil.vmVersion(monitoredVm);
            } finally {
                monitoredHost.detach(monitoredVm);
            }
        } catch (URISyntaxException | MonitorException e) {
            e.printStackTrace();
        }

        System.out.println(
                vmd.id() + '\t' + vmd.displayName() + '\t' + vmVersion +
                 '\t' + isAttachable);
```

```
    }
  }
```

Although classes like `VirtualMachineDescriptor` are part of the exported interface of the `jdk.attach` module (because the class is in the exported package `com.sun.tools.attach`), other classes that we depend on (such as `MonitoredVmUtil` in `sun.jvmstat.monitor`) are not accessible. Fortunately, the tools provide a way to soften the module boundaries and provide access to a nonexported package.

To achieve this, we need to add a switch—`--add-exports`—to force access to the internals of the `jdk.internal.jvmstat` module, which means we are definitely breaking encapsulation by doing this. The resulting compilation command line looks like this:

```
$ javac -d out/wgjd.discovery \
  --add-exports=jdk.internal.jvmstat/sun.jvmstat.monitor=wgjd.discovery \
  wgjd.discovery/module-info.java \
  wgjd.discovery/wgjd/discovery/*.java \
  wgjd.discovery/wgjd/discovery/internal/*
```

The syntax of `--add-exports` is that we must provide the module and package name that we require access to and which module is being granted the access.

4.2 Executing a modular app

Until the arrival of modules, only the following two methods existed to start a Java application:

```
java -cp classes wgjd.Hello
java -jar my-app.jar
```

These should both be familiar to Java programmers as launching a class and the main class from within a JAR file. In modern Java, two more methods of launching programs have been added. We met a new way of launching single-source-file programs in section 1.5.4, and now we're going to meet the fourth mode: launching the main class of a module. The syntax follows:

```
java --module-path mods -m my.module/my.module.Main
```

However, just as for compilation, we may need additional command-line switches. For example, from our earlier example of introspection:

```
$ java --module-path out -m wgjd.discovery/wgjd.discovery.Discovery
Exception in thread "main" java.lang.IllegalAccessError:
  class wgjd.discovery.VMIntrospector (in module wgjd.discovery) cannot
    access class sun.jvmstat.monitor.MonitorException (in module
      jdk.internal.jvmstat) because module jdk.internal.jvmstat does not
        export sun.jvmstat.monitor to module wgjd.discovery
    at wgjd.discovery/wgjd.discovery.VMIntrospector.accept(
    VMIntrospector.java:19)
    at wgjd.discovery/wgjd.discovery.Discovery.main(Discovery.java:26)
```

To prevent this error, we must also provide the encapsulation-breaking switch to the actual program execution as follows:

```
$ java --module-path out \
  --add-exports=jdk.internal.jvmstat/sun.jvmstat.monitor=wgjd.discovery \
  -m wgjd.discovery/wgjd.discovery.Discovery

Java processes:
PID    Display Name    VM Version    Attachable
53407    wgjd.discovery/wgjd.discovery.Discovery    15-ea+24-1168    true
```

If the runtime system can't find the root module we asked for, then we expect to see an exception like this:

```
$ java --module-path mods -m wgjd.hello/wgjd.hello.HelloWorld
Error occurred during initialization of boot layer
java.lang.module.FindException: Module wgjd.hello not found
```

Even this simple error message is showing us that we have new aspects to the JDK, including

- Packages, including `java.lang.module`
- Exceptions, including `FindException`

This shows once again that the modules system really has become an integral part of the execution of every Java program, even if it is not always immediately obvious.

In the next section, we'll briefly introduce the interaction of modules with reflection. We assume that you're already familiar with reflection, but if you're not, feel free to skip this section for now and come back to it after you've read chapter 4, which contains an introduction to class loading and reflection.

4.3 Modules and reflection

In Java 8, developers can use reflection to access almost anything in the runtime. There's even a way to bypass the access control checks in Java and, for example, call private methods on other classes via the so-called `setAccessible()` hack.

As we've already seen, modules change the rules for access control. This also applies to reflection—the intent is that by default, only exported packages should be accessed reflectively.

However, the creators of the modules system realized that sometimes developers want to give reflective access (but not direct access) to certain packages. This requires an explicit permission and can be achieved by using the `opens` keyword to provide reflective-only access to an otherwise internal package. Developers can also specify fine-grained access by using the syntax `opens ... to ...` to allow a named set of packages to be opened reflectively to specific modules, but not more generally.

The previous discussion seems to imply that these types of reflective tricks are now ruled out. The truth is a little more complicated and is best explained via a discussion of the command-line switch `--illegal-access`. This switch comes with three settings—`permit|warn|deny`—and is used to control the strictness of checks on reflection.

The intent of the modules system has always been that over time, the entire Java ecosystem should move toward proper encapsulation, including reflection, and that at some point, the switch will default to `deny` (and will ultimately be removed). This change obviously could not happen overnight—if the reflection switch was suddenly set to `deny`, huge swathes of the Java ecosystem would break and *no one would upgrade*.

However, with the release of Java 17, it is now four years since Java 9 was released and this warning first started to appear. This is, surely, time enough and fair warning. Accordingly, the decision was made in Java 16 to change the default option of `--illegal-access` to `deny` and to remove the option's effect completely in Java 17.

NOTE This change to reflective encapsulation semantics is one reason an application migrating directly from 8 to 17 may see more headaches than one that performs two upgrade hops (8 to 11 and then 11 to 17).

It is still possible to use the `--add-opens` command-line option, or the `Add-Opens` JAR manifest attribute, to open specific packages. This usage may be required for specific libraries or frameworks that have always used reflection and have not yet fully modularized. However, the brute force option to globally reenable access has been removed in Java 17.

One additional useful concept to help this transition is *open modules*. This simple declaration is used to allow for completely open reflective access—it opens all the module's packages for reflection but not compile-time access. This provides simple compatibility with existing code and frameworks but is a looser form of encapsulation. For this reason, open modules are best avoided or used only as a transitional form when migrating to a modular build. In chapter 17, we will discuss the specific case of `Unsafe`, which is a great example to indicate some of the problems with reflection in a modular world.

2.5 Architecting for modules

Modules represent a fundamentally new way of packaging and deploying code. Teams do need to adopt some new practices to get the most out of the new functionality and the architectural benefits. However, the good news is there's no need to start doing that straightaway just to start using modern Java. The traditional, old-school methods using the classpath and JAR files will continue to work until such time as teams are ready to adopt modules wholeheartedly.

In fact, Mark Reinhold (chief architect for Java at Oracle) had this to say about the "need" for applications to adopt modularity.

There is no need to switch to modules.

There has never been a need to switch to modules.

Java 9 and later releases support traditional JAR files on the traditional class path, via the concept of the unnamed module, and will likely do so until the heat death of the universe.

Whether to start using modules is entirely up to you.

If you maintain a large legacy project that isn't changing very much, then it's probably not worth the effort.

—Mark Reinhold, **https://stackoverflow.com/a/62959016**

In an ideal world, modules would be the default for all greenfield apps, but this is proving to be complex in practice, so as an alternative, when migrating, follow a process like this:

1. Upgrade to Java 11 (classpath only).
2. Set an automatic module name.
3. Introduce a *monolithic module* consisting of all code.
4. Break out into individual modules as needed.

Typically, at step 3, way too much implementation code is exposed. This means that quite often, part of the work of step 4 is to create additional packages to house internal and implementation code and to refactor code into them.

If you are still using Java 8 and you aren't ready yet to migrate to a modular build, you can still do the following things to prepare your code for the migration:

- Introduce an automatic module name in MANIFEST.MF.
- Remove split packages from your deployment artifacts.
- Use `jdeps` and Compact Profiles to reduce your footprint of unnecessary dependencies.

To take the first of these, the use of an explicit automatic module name (as we discussed earlier in the chapter) will ease the transition. The automatic module name will be ignored by all versions of Java that do not support modules but will still allow you to reserve a stable name for your library and to move some code out of the unnamed module. It also has the advantage that consumers of your library are prepared for the transition to modules, because you have already advertised the name the module will be using. Let's take a closer look at the other two concrete recommendations.

5.1 Split packages

One common problem that developers encounter when they start to use modules is *split packages*—when two or more separate JARs contain classes belonging to the same package. In a nonmodular application, there is no problem with split packages because neither JAR files nor packages have any particular significance to the runtime. However, in the modular world, a package must belong to only one module and cannot be split.

If an existing application is upgraded to use modules and has dependencies that contain split packages, this will have to be remediated—there just is no way around it. For code that the team controls, this is additional work but not too difficult. One technique is to have a specific artifact (often using a `-all` suffix) that is generated by the build system

alongside the nonmodular versions, with a single JAR containing all parts of the split package.

For external dependencies, remediation can be more complicated. It may be necessary to repackage third-party open source code into a JAR that can be consumed as an automatic module.

5.2 Java 8 Compact Profiles

Compact Profiles are a Java 8 feature. They are runtime environments that are reduced in size that must implement both the JVM and Java language specifications. They were introduced in Java 8 as a useful stepping-stone to the modularity story that would arrive in Java 9.

A Compact Profile must include all classes and packages that are explicitly mentioned in the Java language specification. Profiles are lists of packages, and they are usually identical to the package of the same name in the full Java SE platform. Very few exceptions exist, but they are explicitly called out.

One of the main use cases of profiles is as the basis for a server application or other environment, where deploying unnecessary capabilities is undesirable. For example, historically, a large number of security vulnerabilities were connected to Java's GUI features, especially in Swing and AWT. By choosing not to deploy the packages that implement those features in applications where they are not needed, we can gain a modest amount of additional security, especially for, for example, server applications.

NOTE At one time, Oracle shipped a cut-down JRE (the "Server JRE") that played a very similar role to Compact 1.

Compact 1 is the smallest set of packages on which it is feasible to deploy an application. It contains 50 packages, from the very familiar

- java.io
- java.lang
- java.math
- java.net
- java.text
- java.util

to some perhaps more unexpected packages that nonetheless provide essential classes to modern applications:

- java.util.concurrent.atomic
- java.util.function
- javax.crypto.interfaces
- javax.net.ssl
- javax.security.auth.x500

Compact 2 is significantly larger, containing packages such as those needed for XML, SQL, RMI, and security. Compact 3 is larger still and basically consists of the entire JRE, minus the windowing and GUI components—similar to the `java.se` module.

NOTE All profiles ship the transitive closure of types referred to by `Object` and all types mentioned within the language specification.

The Compact 1 profile is the closest to a minimal runtime, so in some ways it resembles a prototypical form of the `java.base` module. Ideally, if your application or library can be made to run with only Compact 1 as a dependency, then it should.

To help determine whether your app can run with Compact 1 or another profile, the JDK provides `jdeps`. This is a static analysis tool that ships with Java 8 and 11 for examining the dependencies of packages or classes. The tool can be used in a number of different ways, from identifying which profile an application needs to run under, to identifying developer code that makes calls into the undocumented, internal JDK APIs (such as the `sun.misc` classes), through to helping trace transitive dependencies. It can be very helpful for migrations from Java 8 to 11 and works with both JARs and modules. In its simplest form, `jdeps` takes a class or package and provides a brief list of packages that are the dependencies. For example, for the discovery example:

```
$ jdeps Discovery.class
Discovery.class -> java.base
Discovery.class -> jdk.attach
Discovery.class -> not found
   wgjd.discovery          -> com.sun.tools.attach     jdk.attach
   wgjd.discovery          -> java.io                  java.base
   wgjd.discovery          -> java.lang                java.base
   wgjd.discovery          -> java.util                java.base
   wgjd.discovery          -> wgjd.discovery.internal  not found
```

The `-P` switch shows which profile is needed for a class (or package) to run, although, of course, this works only for a Java 8 runtime.

Let's move on to take a quick look at another migration technique that a well-grounded Java developer should be aware of—the use of *multi-release JARs*.

5.3 Multi-release JARs

This new capability allows the construction of a JAR file that can house libraries and components that can work on both Java 8 and modern, modular JVMs. For example, you can depend on library classes that are available only in later versions but can still run on an earlier version by using a fallback and stubbing approach.

To make a multi-release JAR, the following entry must be included in the JAR's manifest file:

```
Multi-Release: true
```

This entry is meaningful to JVMs only from version 9 upward, so if the JAR is used on a Java 8 (or earlier) VM, the multi-release nature will be ignored.

The classes that target post–Java 8 versions are referred to as *variant code* and are stored in a special directory in `META-INF` within the JAR, as shown next:

```
META-INF/versions/<version number>
```

The mechanism works by overriding on a per-class basis. Versions 9 and upward of Java will look for a class that has the exact same name in the `versions` directories as in the main content root. If one is found, the overridden version is used in place of the class in the content root.

NOTE Java class files are stamped with the version number of the Java compiler that created them—the *class file version number*—and code created on a later Java release will not run on an older JVM.

The `META-INF/versions` location is ignored by Java 8 and earlier, so this provides a clever trick to sidestep the fact that some of the code contained in a multi-release JAR has too high a class file version to run on Java 8.

However, this does mean that the API of the class in the content root and any overridden variants must be the same, because they will be linked in exactly the same way for both cases.

Example: Building a multi-release JAR

Let's look at providing an example capability: getting the process ID of the running JVM. Unfortunately, this is somewhat cumbersome on versions of Java before 9 and requires some low-level hackery in the `java.lang.management` package.

Java 11 does provide an API for getting a PID from the Process API, so we want to set up a simple multi-release JAR that will use the simpler API when it is available and fall back to the JMX-based approach only if necessary.

The main class looks like this:

```
public class Main {
   public static void main(String[] args) {
      System.out.println(GetPID.getPid());
   }
}
```

Note that the capability that has a version-dependent implementation has been isolated into a separate class, `GetPID`. The Java 8 version of the code is somewhat verbose, as shown here:

```
public class GetPID {
  public static long getPid() {
    System.out.println("Java 8 version...");                    (1)
    // ManagementFactory.getRuntimeMXBean().getName() returns the name that
    // represents the currently running JVM. On Sun and Oracle JVMs, this
    // name is in the format <pid>@<hostname>.

    final var jvmName = ManagementFactory.getRuntimeMXBean().getName();
    final var index = jvmName.indexOf('@');
    if (index < 1) {
       return 0;
    }

    try {
       return Long.parseLong(jvmName.substring(0, index));
    } catch (NumberFormatException e) {
       return 0;
    }
  }
}
```

❶ We include this line so we can see that this is the Java 8 version.

This requires us to parse a string from a JMX method—and even then our solution is not guaranteed to be portable across JVM implementations. By contrast, Java 9 and later provide a much simpler standard method in the API, as shown in the following code snippet:

```
public class GetPID {
  public static long getPid() {
    // Use the ProcessHandle API, new in Java 9 ...
    var ph = ProcessHandle.current();
    return ph.pid();
  }
}
```

Because the `ProcessHandle` class is in the package `java.lang`, we don't even need an `import` statement.

We now need to arrange the multi-release JAR so that the Java 11 code is included in the JAR and is used in preference to the fallback version, if the JVM has a high enough version. One suitable code layout looks like this:

```
.
└── src
    ├── main
    │   └── java
    │       └── wgjd2ed
    │           ├── Main.java
    │           └── GetPID.java
    └── versions
        └── 11
            └── java
                └── wgjd2ed
                    └── GetPID.java
```

The main part of the codebase needs to be compiled with Java 8, and then the post-Java 8 code is compiled afterward with a different Java version, before being packaged into a multi-release JAR "by hand" (i.e., using the command-line `jar` tool directly).

NOTE This code layout uses the convention that the Maven and Gradle build tools follow, which we'll meet properly in chapter 11.

Let's compile the code from the command line using JDK version `javac` but targeting our output to Java 8 via the `--release` flag:

```
$ javac --release 8 -d out src/main/java/wgjd2ed/*.java
```

Next, we build the Java 11 targeted code with a separate output directory for the variant code `out-11`:

```
$ javac --release 11 -d out-11 versions/11/java/wgjd2ed/GetPID.java
```

We also need a MANIFEST.MF file, but we can use the (Java 11) `jar` tool to automatically construct what we need as follows:

```
$ jar --create --release 11 \
    --file pid.jar --main-class=wgjd2ed.Main
    -C out/ . \
    -C out-11/ .
```

This creates a multi-release JAR, which is also runnable (with `Main` being the entry point class). Running the JAR gives the next output on Java 11:

```
$ java -version
openjdk version "11.0.3" 2019-04-16
OpenJDK Runtime Environment AdoptOpenJDK (build 11.0.3+7)
OpenJDK 64-Bit Server VM AdoptOpenJDK (build 11.0.3+7, mixed mode)

$ java -jar pid.jar
13855
```

and on Java 8:

```
$ java -version
openjdk version "1.8.0_212"
OpenJDK Runtime Environment (AdoptOpenJDK)(build 1.8.0_212-b03)
OpenJDK 64-Bit Server VM (AdoptOpenJDK)(build 25.212-b03, mixed mode)

$ java -jar pid.jar
```

```
Java 8 version...
13860
```

Note the extra banner we added to the Java 8 version so you can distinguish between the two cases and be sure that the two different classes are actually being run. For real-world use cases for multi-release JARs, we would want the code to either perform the same in both cases (if we're shimming a capability back to Java 8) or to fail in a graceful or predictable way if run on a JVM that doesn't support a capability.

One important architectural pattern that we recommend following is isolating JDK version-specific code into a package or group of packages, depending on how extensive the capability is.

Some basic guidelines and principles for the project follow:

- The main codebase must be able to be built with Java 8.
- Java 11 portion must be built with Java 11.
- Java 11 portion must be in a separate code root, isolated from the main build.
- The end result should be a single JAR.
- Keep the build configuration as simple as possible.
- Consider making the multi-release JAR modular as well.

The last point is especially important, and this continues to be true for more complex projects that will inevitably end up requiring a proper build tool, rather than just `javac` and `jar`.

2.6 Beyond modules

To conclude the chapter, let's take a quick look at what lies beyond modules. Recall that the entire point of modules was to introduce a missing abstraction into the Java language: the idea of deployment units with dependency guarantees that can be relied upon by the source code compiler and runtime.

This idea of modular dependency information that can be trusted has a number of applications to the modern world of deployable software. In Java, modules have seen slow but steady adoption as the tooling and ecosystem comes to fully support them and the benefits that they enable have become better understood.

Let's conclude this chapter by meeting a new capability that was added to the platform along with modules—*JLink*. This is the ability to package a reduced Java runtime along

with an application. This provides the following benefits to applications that make use of it:

- Package the application and JVM into a single, self-contained directory.
- Reduce the footprint and overall download size of the application and JRE bundle.
- Reduce support overhead, because there is no need to debug interactions between a Java application and a host-installed JVM.

The self-contained directories that `jlink` produces can easily be packaged as deployable artifacts (such as a Linux `.rpm` or `.deb`, a Mac `.dmg`, or a Windows `.msi`), providing a simple installation experience for modern Java applications.

In some ways, the Compact Profiles technology in Java 8 provides an early version of JLink, but the version that arrived with modules is far more useful and comprehensive. As an example, we're going to reuse the discovery example from earlier in the chapter. This has a simple module-info.java:

```
module wgjd.discovery {
  exports wgjd.discovery;

  requires java.instrument;
  requires java.logging;
  requires jdk.attach;
  requires jdk.internal.jvmstat;
}
```

This can be built into a JLink bundle via a command like this:

```
$ jlink --module-path $JAVA_HOME/jmods/:out  --output bundle/ --add-modules
  wgjd.discovery
```

In our simple example, we have produced a JLink bundle that can be delivered as a TAR ball or packaged into a Linux package (such as .deb or .rpm). We can actually take this one step further and use *static compilation* to convert such a bundle into a native executable, but a full discussion of this is outside the scope of this book.

We should sound a word of caution: JLink is a great piece of technology, but it has some important limitations that you should be aware of:

- It will work only with an application with fully modularized dependencies.

- It does not work with nonmodular code.
- Even automatic modules are not sufficient.

This is because, to be absolutely sure that all of the necessary parts of the JRE are included in the bundle, JLink relies upon the strongly declarative information in the module graph and therefore requires a `module-info.class` for each dependency. Without this information, building a reduced JRE is very likely to be unsafe.

Unfortunately, in the real world, many libraries that applications depend upon are still not fully modularized. This sharply reduces the usefulness of JLink. To solve this problem, toolmakers have developed plugins to repackage and synthesize "true" modules from unmodularized libraries. We'll discuss these in chapter 11. However, to use these tools requires the use of a build system. This means we will defer real-world examples of JLink until later on, when we meet the build tools in chapter 11.

Summary

- Modules are a new concept in Java. They group packages and provide metadata about the whole unit, its dependencies, and its public interfaces. These constraints are then enforced by the compiler and runtime.
- Modules are *not* a deployment construct (e.g., a different file format). Modularized libraries and applications can still be distributed via JAR files and downloaded by standard build tools.
- Moving to modules requires changes in how we develop our Java applications.
  - New syntax in the module-info.java file controls how classes and methods are exposed within the module system.
  - Class loading is aware of the restrictions a module defines and handles loading nonmodular code.
  - Building with modules requires new command-line flags and a change to the standard layout for a Java project.
- Modules provide a number of benefits in return for this work.
  - Because of the more granular controls, modules are a fundamentally better way to architect applications for modern deployment and future maintainability.
  - Modules are key to reducing footprint, especially in containers.
  - Modules pave the way for other new capabilities (such as static compilation).
- Migrating to modules can be challenging, especially for legacy monolithic applications. Even three years after the first modular runtime was released, adoption remains patchy and incomplete.
- Tools such as multi-release JARs and Compact Profiles can help prepare existing Java 8 projects to integrate with the modular ecosystem, even if they can't move now.

Previous chapter

< 1 Introducing modern Java

Next chapter

3 Java 17 >